

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 3

Roll no: 54 PRN No: 12414148 Name of Student: Sanika Kulkarni

Subject Name: Deep Learning

Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

1. Start the program.
2. Import the required libraries.
3. Load the dataset.
4. Split the dataset into training and testing sets.
5. Preprocess the data (reshape if required and normalize the feature values).
6. Build the neural network model by adding the input, hidden, and output layers.
7. Compile the model by selecting the optimizer, learning rate, loss function, and evaluation metrics.
8. Train the model using the training dataset for the specified number of epochs.
9. Evaluate the trained model using the testing dataset.
10. Repeat the training process with different learning rates and compare the model performance.
11. Repeat the training process with different numbers of epochs and compare the model performance.
12. Plot graphs to analyze accuracy and loss during training and validation.
13. Display the final performance metrics and compare the results.
14. Stop the program

Code:

Using MNIST dataset: Dataset of 70,000 grayscale images of handwritten digits (0–9) used for image classification.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd
(x_train,y_train),(x_test,y_test)=tf.keras.datasets.mnist.load_data()
x_train=x_train.reshape(-1,784)/255.0
x_test=x_test.reshape(-1,784)/255.0
def create_model(lr):
    m=Sequential([Dense(128,activation='relu',input_shape=(784,)),
                  Dense(64,activation='relu'),
                  Dense(10,activation='softmax')])
    m.compile(optimizer=Adam(learning_rate=lr),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m
learning_rates=[0.1,0.01,0.001]
results=[]
for lr in learning_rates:
    model=create_model(lr)
    model.fit(x_train,y_train,epochs=5,batch_size=128,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    results.append([lr,acc])
df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
print(df)
plt.plot(df['Learning Rate'],df['Accuracy'],marker='o')
plt.xscale('log'); plt.grid(); plt.show()
epochs_list=[5,10,20]
res=[]
for ep in epochs_list:
    model=create_model(0.001)
    h=model.fit(x_train,y_train,validation_split=0.2,epochs=ep,batch_size=128,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    res.append([ep,acc])
edf=pd.DataFrame(res,columns=['Epochs','Accuracy'])
print(edf)
plt.plot(edf['Epochs'],edf['Accuracy'],marker='o')
plt.grid(); plt.show()
```

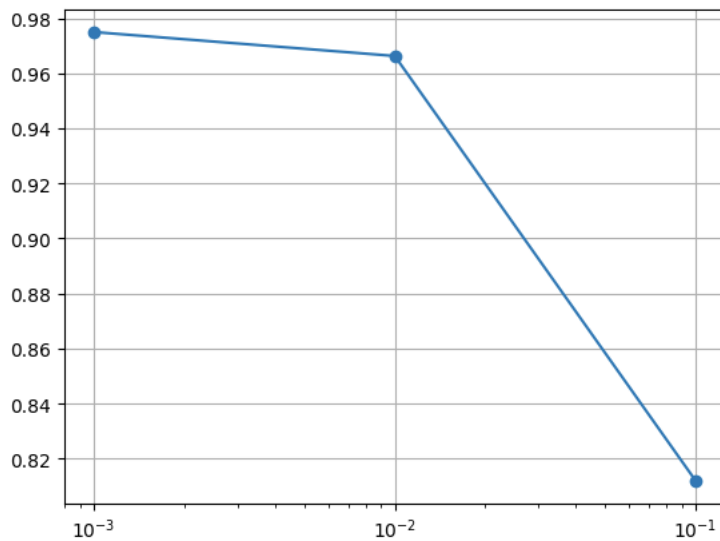
```

model=create_model(0.001)
history=model.fit(x_train,y_train,validation_split=0.2,epochs=20,batch_size=128)
plt.plot(history.history['accuracy']); plt.plot(history.history['val_accuracy']);
plt.legend(['Train','Val']); plt.show()
plt.plot(history.history['loss']); plt.plot(history.history['val_loss']);
plt.legend(['Train','Val']); plt.show()
print(model.evaluate(x_test,y_test))

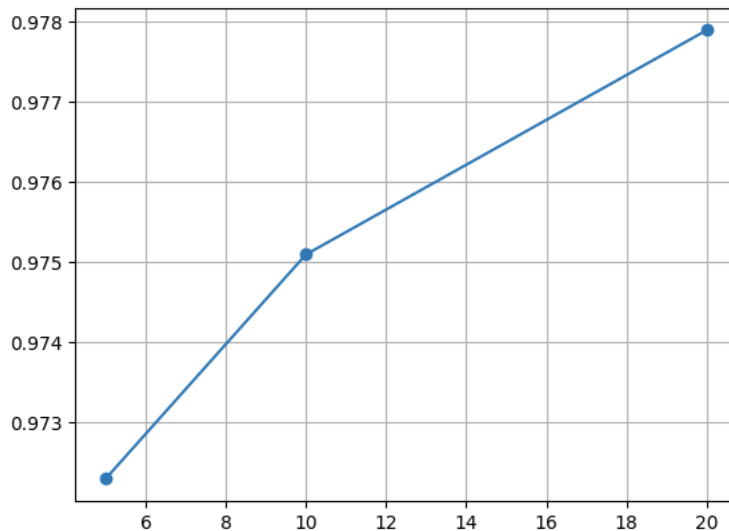
```

Output:

	Learning Rate	Accuracy
0	0.100	0.8120
1	0.010	0.9663
2	0.001	0.9750



	Epochs	Accuracy
0	5	0.9723
1	10	0.9751
2	20	0.9779



```

Epoch 1/20
375/375 4s 9ms/step - accuracy: 0.8914 - loss: 0.3816 - val_accuracy: 0.9477 - val_loss: 0.1820
Epoch 2/20
375/375 2s 5ms/step - accuracy: 0.9545 - loss: 0.1557 - val_accuracy: 0.9628 - val_loss: 0.1296
Epoch 3/20
375/375 2s 5ms/step - accuracy: 0.9679 - loss: 0.1084 - val_accuracy: 0.9688 - val_loss: 0.1078
Epoch 4/20
375/375 3s 5ms/step - accuracy: 0.9758 - loss: 0.0812 - val_accuracy: 0.9672 - val_loss: 0.1133
Epoch 5/20
375/375 2s 5ms/step - accuracy: 0.9813 - loss: 0.0636 - val_accuracy: 0.9696 - val_loss: 0.1005
Epoch 6/20
375/375 2s 5ms/step - accuracy: 0.9833 - loss: 0.0531 - val_accuracy: 0.9735 - val_loss: 0.0891
Epoch 7/20
375/375 3s 9ms/step - accuracy: 0.9874 - loss: 0.0429 - val_accuracy: 0.9734 - val_loss: 0.0861
Epoch 8/20
375/375 2s 5ms/step - accuracy: 0.9892 - loss: 0.0350 - val_accuracy: 0.9737 - val_loss: 0.0910
Epoch 9/20
375/375 2s 5ms/step - accuracy: 0.9918 - loss: 0.0275 - val_accuracy: 0.9699 - val_loss: 0.1096
Epoch 10/20
375/375 2s 5ms/step - accuracy: 0.9930 - loss: 0.0236 - val_accuracy: 0.9712 - val_loss: 0.1049
Epoch 11/20
375/375 2s 5ms/step - accuracy: 0.9937 - loss: 0.0209 - val_accuracy: 0.9739 - val_loss: 0.0975
Epoch 12/20
375/375 2s 5ms/step - accuracy: 0.9948 - loss: 0.0175 - val_accuracy: 0.9758 - val_loss: 0.0941
Epoch 13/20
375/375 3s 8ms/step - accuracy: 0.9965 - loss: 0.0135 - val_accuracy: 0.9743 - val_loss: 0.1015
Epoch 14/20
375/375 4s 5ms/step - accuracy: 0.9956 - loss: 0.0141 - val_accuracy: 0.9760 - val_loss: 0.1004
Epoch 15/20
375/375 2s 5ms/step - accuracy: 0.9956 - loss: 0.0134 - val_accuracy: 0.9746 - val_loss: 0.1049
Epoch 16/20
375/375 2s 5ms/step - accuracy: 0.9976 - loss: 0.0088 - val_accuracy: 0.9745 - val_loss: 0.1146
Epoch 17/20
375/375 2s 5ms/step - accuracy: 0.9971 - loss: 0.0098 - val_accuracy: 0.9757 - val_loss: 0.1109
Epoch 18/20
375/375 3s 8ms/step - accuracy: 0.9969 - loss: 0.0098 - val_accuracy: 0.9743 - val_loss: 0.1167
Epoch 19/20
375/375 2s 6ms/step - accuracy: 0.9983 - loss: 0.0062 - val_accuracy: 0.9768 - val_loss: 0.1146
Epoch 20/20
375/375 2s 5ms/step - accuracy: 0.9991 - loss: 0.0039 - val_accuracy: 0.9767 - val_loss: 0.1139

```

